Topic and contents

UNSW, School of Mathematics and Statistics

Profs Josef Dick and Kassem Mustapha

MATH3311/MATH5335

Topic 07 – Implied Volatility, Nonlinear Equations

Options

A European call (put) option on an underlying asset gives the owner the right (but not the obligation) to buy (sell) that asset for a nominated strike price (or exercise price) K at a nominated expiry time T. Let

$$S = \text{price}, \qquad t = \text{time},$$
$$r = \text{risk-free interest rate}, \qquad \sigma = \text{volatility of the asset price}.$$

For the call option value $c(S,t)$, the Black and Scholes (1973) satisfies

$$\frac{\partial c}{\partial t} + \tfrac{1}{2}\sigma^2 S^2 \frac{\partial^2 c}{\partial S^2} + rS\frac{\partial c}{\partial S} = rc$$

for $0 < S < \infty$ and $0 < t < T$, with boundary conditions

$$c(0,t) = 0 \quad \text{and} \quad c(S,t) \sim S \text{ as } S \to \infty,$$

and with the "terminal" condition at T (one may think of it as an "initial" condition posed backward in time),

$$c(S,T) = \max(S-K, 0).$$

Black and Scholes formula

This terminal boundary-value problem has an explicit solution

$$c(S,t) = SN(d_1) - Ke^{-r(T-t)}N(d_2),$$

where N is the CDF of the standard normal distribution:

$$N(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-\xi^2/2}\, d\xi,$$

$$d_1 = \frac{\log(S/K) + r(T-t)}{\sigma\sqrt{T-t}} + \tfrac{1}{2}\sigma\sqrt{T-t},$$

$$d_2 = \frac{\log(S/K) + r(T-t)}{\sigma\sqrt{T-t}} - \tfrac{1}{2}\sigma\sqrt{T-t} = d_1 - \sigma\sqrt{T-t}.$$

Thus, if we know S, $T-t$ (time to maturity), r, σ, and K, then we can calculate c.

Put-call parity: for a European call c and put p,

$$c + Ke^{-r(T-t)} = p + S.$$

Implied volatility

In practice, estimating σ is difficult but we can look up the market value of the call option, c_{mkt}.

The implied volatility σ^* is the value of σ for which $c = c_{\text{mkt}}$. Thus, define

$$f(\sigma) = SN(d_1) - Ke^{-r(T-t)}N(d_2) - c_{\text{mkt}},$$

and then, σ^* is the solution of the nonlinear equation $f(\sigma) = 0$.

- Questions: Do we have a unique solution? How to solve this nonlinear equation efficiently?

Existence of a solution of $f(\sigma) = 0$.

- Intermediate value theorem: If f is continuous on $[a, b]$ and $f(a)f(b) < 0$ then there exists $\alpha \in (a, b)$ such that $f(\alpha) = 0$.
- As $f(\sigma)$ is continuous on $(0, \infty)$, at least one solution σ^* exists if $\lim_{\sigma\to\infty} f(\sigma) \geq 0$ and $\lim_{\sigma\to 0^+} f(\sigma) \leq 0$, or vise-versa.

As $\sigma \to \infty$, $d_1 \to \infty$ and $d_2 \to -\infty$. Thus, $N(d_1) \to 1$ and $N(d_2) \to 0$. Therefore,

$$\lim_{\sigma\to\infty} f(\sigma) = S - c_{\text{mkt}}.$$

Recall that

$$d_1 = \frac{\log(S/K) + r(T-t)}{\sigma\sqrt{T-t}} + \tfrac{1}{2}\sigma\sqrt{T-t} \quad \text{and} \quad d_2 = d_1 - \sigma\sqrt{T-t}\,.$$

As $\sigma \to 0^+$:

- $d_1,\ d_2 \to 0$ if $\log(S/K) + r(T-t) = 0$, i.e., $S = Ke^{-r(T-t)}$. Then,

$$N(d_1) = N(d_2) = 1/2 \Longrightarrow \lim_{\sigma\to 0^+} f(\sigma) = S/2 - Ke^{-r(T-t)}/2 - c_{\text{mkt}}.$$

- $d_1,\ d_2 \to \infty$ if $\log(S/K) + r(T-t) > 0$, i.e., $S > Ke^{-r(T-t)}$. Then

$$N(d_1) = N(d_2) = 1 \Longrightarrow \lim_{\sigma\to 0^+} f(\sigma) = S - Ke^{-r(T-t)} - c_{\text{mkt}}.$$

- $d_1,\ d_2 \to -\infty$ if $\log(S/K) + r(T-t) < 0$, i.e., $S < Ke^{-r(T-t)}$. Then

$$N(d_1) = N(d_2) = 0 \Longrightarrow \lim_{\sigma\to 0^+} f(\sigma) = -c_{\text{mkt}}.$$

Therefore,

$$\lim_{\sigma\to 0^+} f(\sigma) = \max(S - Ke^{-r(T-t)}, 0) - c_{\text{mkt}}.$$

Now, it is clear that

$$\lim_{\sigma \to 0^+} f(\sigma) \leq \lim_{\sigma \to \infty} f(\sigma),$$

and hence at least one solution σ^* exists if

$$\max(S - Ke^{-r(T-t)}, 0) - c_{\text{mkt}} \leq 0 \leq S - c_{\text{mkt}},$$

or, equivalently, if

$$\max(S - Ke^{-r(T-t)}, 0) \leq c_{\text{mkt}} \leq S.$$

We show next that $f'(\sigma) > 0$ on $(0, \infty)$. This implies that f is increasing on $(0, \infty)$, and consequently the nonlinear equation $f(\sigma) = 0$ has a unique solution σ^*.

$$f(\sigma) = SN(d_1) - Ke^{-r(T-t)}N(d_2) - c_{\text{mkt}}$$

$$\Longrightarrow f'(\sigma) = SN'(d_1)d_1' - Ke^{-r(T-t)}N'(d_2)d_2'.$$

Recall that

$$N(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-\xi^2/2}\, d\xi,$$

and

$$d_1 = \frac{\log(S/K) + r(T-t)}{\sigma\sqrt{T-t}} + \tfrac{1}{2}\sigma\sqrt{T-t}, \quad d_2 = d_1 - \sigma\sqrt{T-t}.$$

Hence $N'(d_1) = \frac{1}{\sqrt{2\pi}} e^{-d_1^2/2} > 0$ and

$$N'(d_2) = \frac{1}{\sqrt{2\pi}} e^{-(d_1 - \sigma\sqrt{T-t})^2/2} = N'(d_1) e^{d_1\sigma\sqrt{T-t} - \frac{1}{2}\sigma^2(T-t)}$$

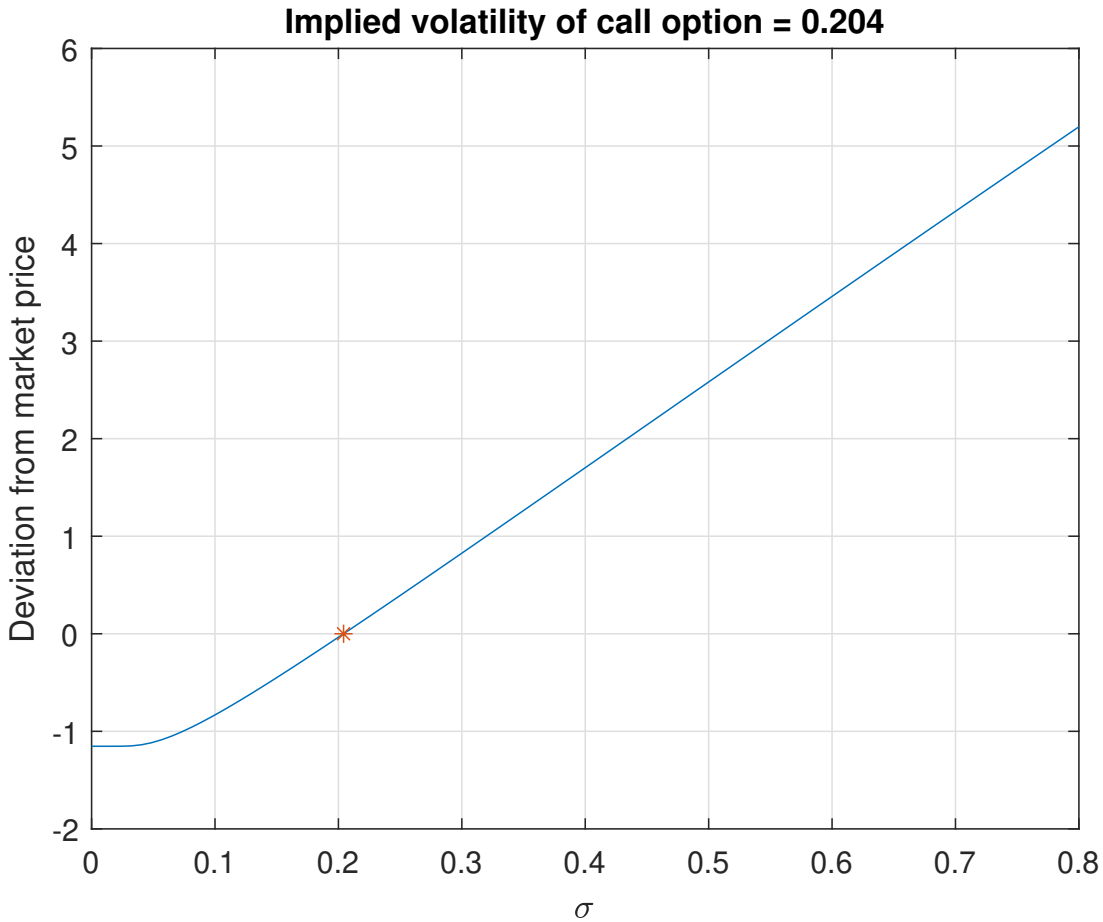
$$= N'(d_1) e^{\log(S/K) + r(T-t) + \frac{1}{2}\sigma^2(T-t) - \frac{1}{2}\sigma^2(T-t)} = \frac{S}{K} N'(d_1) e^{r(T-t)}.$$

Therefore,

$$f'(\sigma) = SN'(d_1)d_1' - SN'(d_1)d_2' = SN'(d_1)\,(d_1 - d_2)' = SN'(d_1)\sqrt{T-t} > 0.$$

Example

$T - t = 6$ months, $r = 5.3\%$ per annum, $S = \$32.75$, $K = \$32$, $c_{\text{mkt}} = \$2.74$.



MATLAB `impvoln.m`, Python `impvoln.py`

Taylor polynomials

Definition (Taylor polynomial)

Let $f \in \mathcal{C}^{n+1}[a,b]$ (all derivatives up to $n+1$ are continuous on $[a,b]$). Then, for $x \in (a,b)$ and $x+h \in [a,b]$,

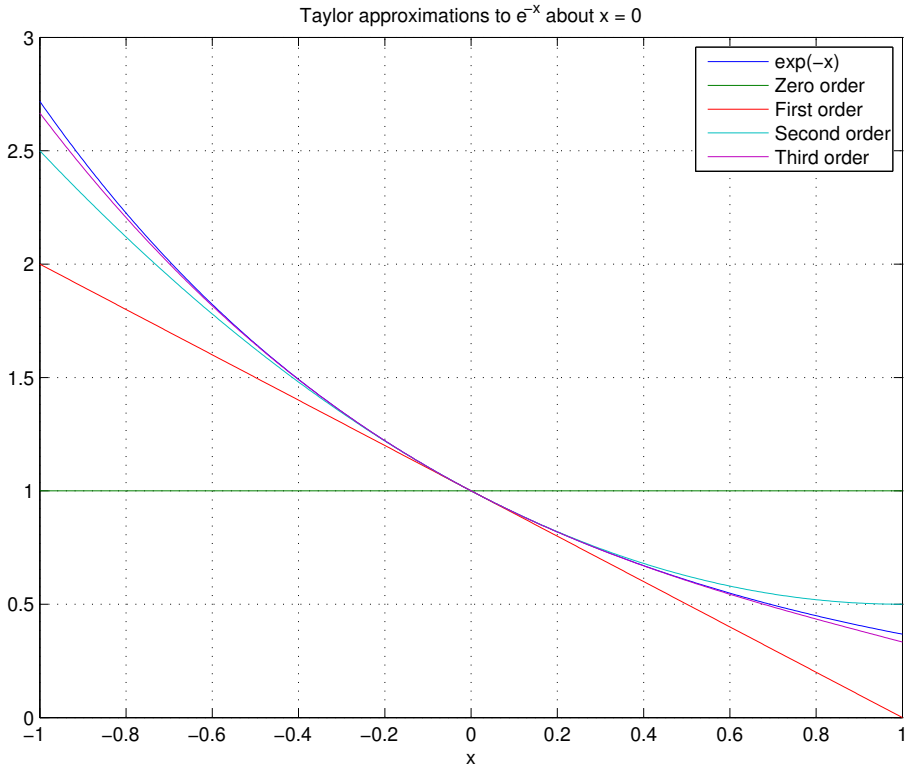
$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + h^2 \frac{f''(x)}{2!} + \cdots + h^n \frac{f^{(n)}(x)}{n!} + R_n(x;h) \\ &= T_n(x;h) + R_n(x;h). \end{aligned}$$

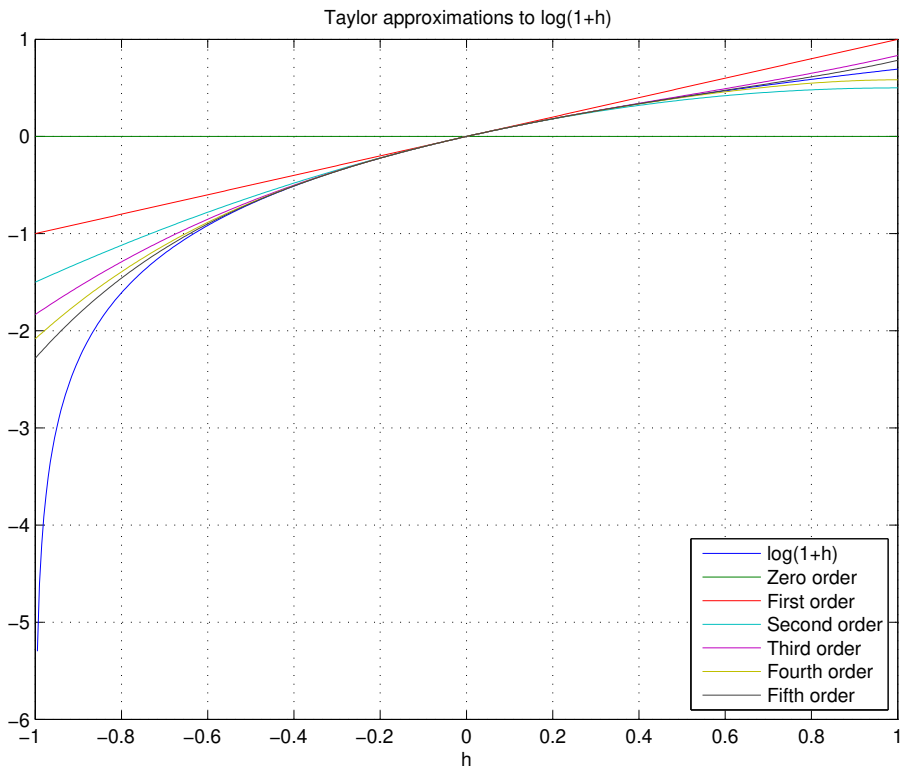
- Taylor polynomial of degree n (in h): $T_n(x;h) = \sum_{k=0}^{n} h^k \frac{f^{(k)}(x)}{k!}$.
- Remainder term $R_n(x;h) = \frac{h^{n+1}}{(n+1)!} f^{(n+1)}(\zeta) = O(h^{n+1})$ for some $\zeta \in (a,b)$.
- If $f \in \mathcal{C}^m[a,b]$ for some $1 \le m \le n+1$, then the remainder term is $O(h^m)$.

Taylor approximations

Example (Taylor polynomial approximations)

- $\exp(-x) = 1 - x + \frac{x^2}{2} - \frac{x^3}{6} + O(x^4)$ `taylor1.m`, `taylor1.py`
- $\log(1+h) = h - \frac{h^2}{2} + \frac{h^3}{3} - \frac{h^4}{4} + \frac{h^5}{5} + O(h^6)$ `taylor2.m`, `taylor2.py`





Forward-backward difference approximations of $f'(x)$

- Assume $f \in \mathcal{C}^2[a,b]$, $x \in (a,b)$ and $x+h \in [a,b]$.
- From Taylor series, with $h > 0$, we have

$$f(x \pm h) = f(x) \pm hf'(x) + O(h^2)$$

Rearranging, we get

$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h) \approx \frac{f(x+h) - f(x)}{h},$$

forward approximation with $O(h)$ error (the difference quotient is taken in the *forward* direction). Or

$$f'(x) = \frac{f(x) - f(x-h)}{h} + O(h) \approx \frac{f(x) - f(x-h)}{h},$$

backward approximation with $O(h)$ error (the difference quotient is taken in the backward direction).

- The truncation error is $O(h)$.

Central difference approximations of $f'(x)$

- Assume $f \in \mathcal{C}^3[a,b]$, $x \in (a,b)$ and $x \pm h \in [a,b]$.
- From the Taylor expansion,

$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + O(h^3), \\ f(x-h) &= f(x) - hf'(x) + \frac{h^2}{2}f''(x) + O(h^3). \end{aligned}$$

- Then

$$f(x+h) - f(x-h) = 2hf'(x) + O(h^3).$$

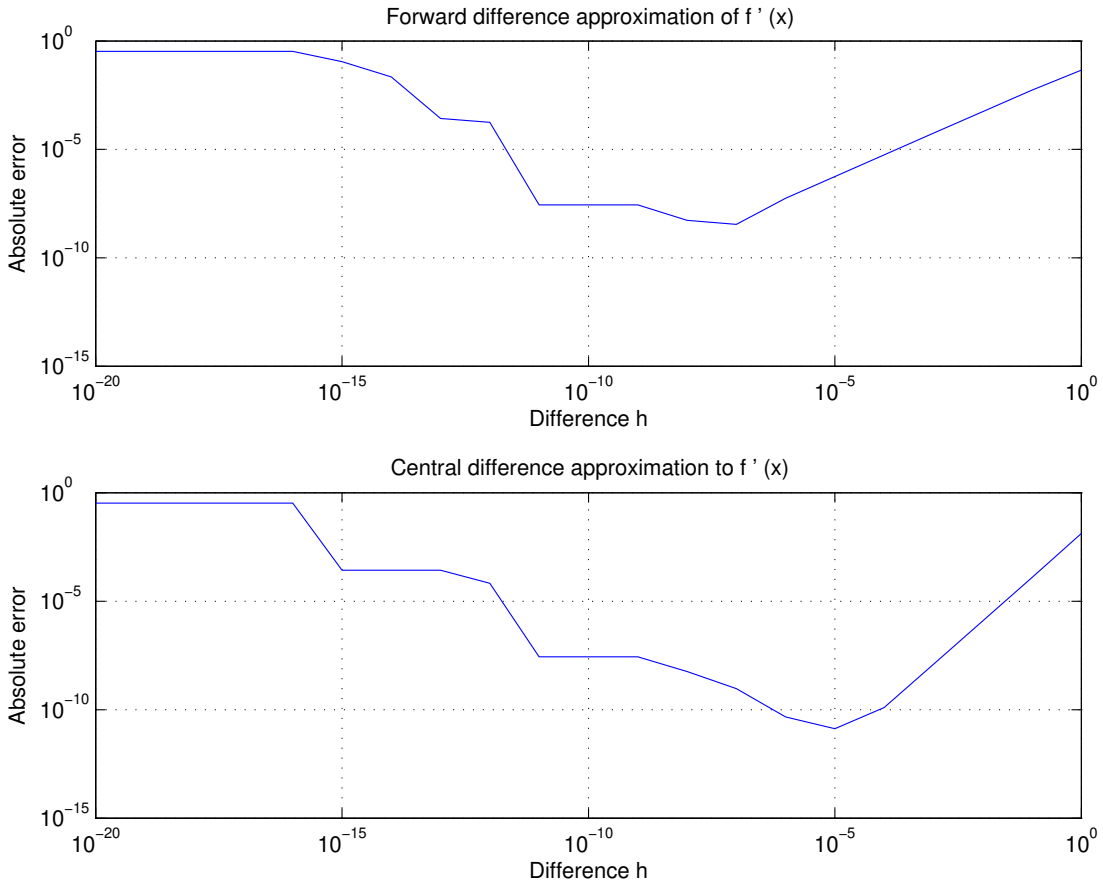
Rearranging, we get the central difference approximation:

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

  - The truncation error is $O(h^2)$.

Example of difference approximations to $f'(x)$

Example: difference approximations to $f'(x)$ for $f(x) = \log(1+x)$ at $x = 2$; the exact value is $f'(2) = 1/3$.



MATLAB M-file `fdiff.m`, Python M-file `fdiff.py`

Truncation, rounding, and optimal stepsize

- Truncation error $TE(h)$ comes from mathematical approximation:
  - Forward or backward difference: $O(h)$.
  - Central difference: $O(h^2)$.
- For h sufficiently small, $x \pm h$ and x may be stored as the same number. In this case, the forward, backward, or central difference approximations of the derivative may fail (not accurate).
- Rounding error $RE(h)$ comes from computer arithmetic:
  - First derivative approximations (forward, backward, or central differences): $RE(h) = O(\epsilon/h)$. Recall that the rounding error in storing $f(x)$ is about $\epsilon|f(x)|$.
  - Rounding error grows as $h \to 0$.
- Total Error is approximately $E(h) = TE(h) + RE(h)$. Choose the optimal stepsize h by balancing both errors to minimize $E(h)$:
  - Forward (or backward) difference: $E(h) = O(h) + O(\epsilon/h)$, optimal $h \approx \epsilon^{1/2}$.
  - Central difference: $E(h) = O(h^2) + O(\epsilon/h)$, optimal $h \approx \epsilon^{1/3}$.

Second central difference approximation to $f''(x)$

- Assume that $f \in \mathcal{C}^4[a,b]$ and $x \in (a,b)$. From the Taylor expansion,

$$\begin{aligned} f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{3!}f'''(x) + O(h^4), \\ f(x-h) &= f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{3!}f'''(x) + O(h^4). \end{aligned}$$

- Then

$$f(x+h) + f(x-h) = 2f(x) + h^2 f''(x) + O(h^4).$$

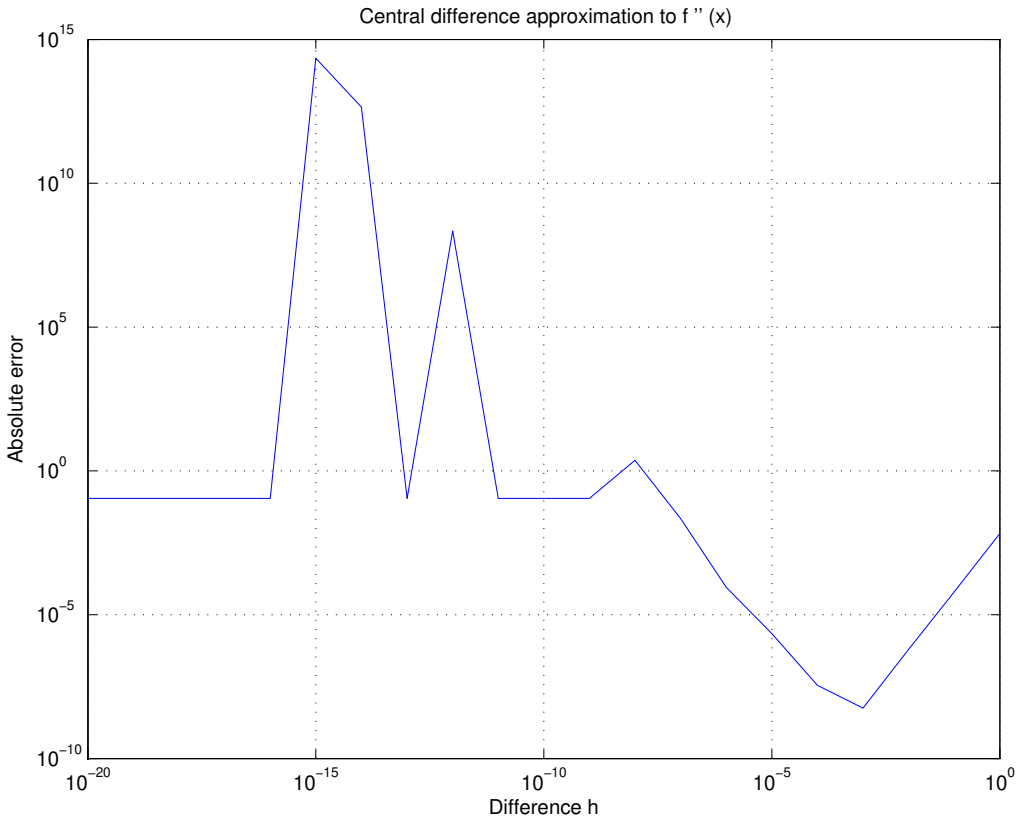
Rearranging, we get the second central difference approximation:

$$\begin{aligned} f''(x) &= \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + O(h^2) \\ &\approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}. \end{aligned}$$

  - The truncation error is $O(h^2)$.

Central difference approximation – optimal stepsize

- Total error = Truncation error + Rounding error; $E(h) = O(h^2) + O(\epsilon/h^2)$.
- Minimizing $E(h)$ by balancing both errors gives the optimal stepsize $h \approx \epsilon^{1/4}$.



Nonlinear equations

Aim: use an iterative process to solve the nonlinear equation

$$f(x) = 0. \tag{1}$$

Iterative process:

- Typically, we start from an initial guess $x^{(0)}$, then generate $x^{(1)}, x^{(2)}, \ldots$ converging to a solution x^*, that is,

$$x^{(k)} \to x^* \quad \text{as } k \to \infty, \quad \text{with } f(x^*) = 0.$$

  If f is continuous at x^*, then this implies $f(x^{(k)}) \to 0$ as $k \to \infty$.
- Ideally, we want $x^{(k)}$ to converge to x^* rapidly, and each iterate $x^{(k)}$ to be easy and cheap to compute.

Order of convergence

Definition (Order of convergence)

Denote the error in the kth iterate $x^{(k)}$ by $e^{(k)} = x^{(k)} - x^*$. The order of convergence is the largest ν such that the limit

$$\beta = \lim_{k \to \infty} \frac{\left|e^{(k+1)}\right|}{\left|e^{(k)}\right|^{\nu}}$$

exists and is finite.

1- The larger the value of ν, the faster the method converges.

2- For linear convergence, that is, for $\nu = 1$, we have

$$|e^{(k+1)}| \approx \beta |e^{(k)}| \quad \text{for large} \quad k.$$

So, we need $\beta < 1$, and the closer β is to zero, the better.

Newton's method

By Taylor's theorem,

$$0 = f(x^*) \approx f(x^{(k)}) + f'(x^{(k)})(x^* - x^{(k)}) \quad \text{and thus}$$

$$x^* \approx x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}.$$

This formula motivates the definition of Newton's method:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})} \quad \text{for } k = 0, 1, 2, \ldots.$$

For the implied volatility example, we find

k	σ_k	$f(\sigma_k)$
0	0.2000000000000000	-3.54e-02
1	0.2041832820973033	4.28e-05
2	0.2041782248699553	6.00e-11
3	0.2041782248628699	-1.78e-15
4	0.2041782248628701	1.78e-15

so $\sigma^* \approx 0.20417822486287$.

Newton's method (continued)

Newton's method is quadratically convergent if the initial guess $x^{(0)}$ is sufficiently close to x^*. If $x^{(0)}$ is too far away from x^*, then Newton's method may fail to converge.

Theorem

Assume that $f(x^*) = 0$, that f' and f'' are continuous on a neighborhood $B_\delta(x^*) := [x^* - \delta, x^* + \delta]$, and that $f' \neq 0$ on $B_\delta(x^*)$. Then, for $x^{(0)} \in B_\delta(x^*)$, the Newton iterates $x^{(1)}, x^{(2)}, \ldots$ converge to x^*, and

$$\lim_{k\to\infty} \frac{|e^{(k+1)}|}{|e^{(k)}|^2} = \frac{|f''(x^*)|}{2|f'(x^*)|}.$$

Thus, $\nu = 2$ and $\beta = \frac{1}{2}|f''(x^*)/f'(x^*)|$.

Note: From calculus, for $x \in B_\delta(x^*)$, we have $|f''(x)/f'(x)| \leq M$ for some $M > 0$. We may assume that $\delta \leq 1/M$.

Sketch of proof: From Taylor's theorem, for some ξ_k between x^* and $x^{(k)}$,

$$0 = f(x^*) = f(x^{(k)}) + f'(x^{(k)})(x^* - x^{(k)}) + \frac{1}{2}f''(\xi_k)(x^* - x^{(k)})^2.$$

Divide by $f'(x^{(k)})$ to get

$$\frac{f(x^{(k)})}{f'(x^{(k)})} + x^* - x^{(k)} = -\frac{f''(\xi_k)}{2f'(x^{(k)})}(x^* - x^{(k)})^2, \quad \text{for} \quad k \geq 0.$$

Since $x^{(k+1)} = x^{(k)} - \dfrac{f(x^{(k)})}{f'(x^{(k)})}$, $x^* - x^{(k+1)} = -\dfrac{f''(\xi_k)}{2f'(x^{(k)})}(x^* - x^{(k)})^2$.

- Let $\delta_k = \frac{\delta}{2^k}$. Since $\delta_0 = \delta$, $x^{(0)} \in B_{\delta_0}$. So, $\xi_0 \in B_{\delta_0}(x^*)$ and
$$|x^* - x^{(1)}| \leq M\delta^2/2 \leq \delta/2 \Longrightarrow x^{(1)} \in B_{\delta_1}(x^*).$$
- By induction, $x^{(k)} \in B_{\delta_k}(x^*)$ for $k \geq 1$. Since $f' \neq 0$ on $B_\delta(x^*)$, we have $f'(x^{(1)}) \neq 0$, $f'(x^{(2)}) \neq 0, \ldots$.

Therefore, $x^{(1)}, x^{(2)}, \ldots$ converges to x^*, and

$$\lim_{k\to\infty} \frac{|e^{(k+1)}|}{|e^{(k)}|^2} = \lim_{k\to\infty} \frac{|f''(\xi_k)|}{2|f'(x^{(k)})|} = \frac{|f''(x^*)|}{2|f'(x^*)|}.$$

Secant method

We define the divided difference

$$f[a,b] = \frac{f(b) - f(a)}{b - a},$$

which equals the slope of the secant line through the points $\big(a, f(a)\big)$ and $\big(b, f(b)\big)$. Since

$$f[x^{(k-1)}, x^{(k)}] \approx f'(x^{(k)}),$$

we may modify Newton's method to obtain the secant method:

$$x^{(k+1)} = x^{(k)} - \frac{f\big(x^{(k)}\big)}{f[x^{(k-1)}, x^{(k)}]} \quad \text{for } k \geq 1.$$

Here, we need two initial guesses $x^{(0)}$ and $x^{(1)}$, but we do not need to evaluate f'.

Secant method (continued)

If the initial guesses are sufficiently close to x^*, then the secant iterates $x^{(2)}, x^{(3)}, \ldots$ converge to x^*, with

$$\lim_{k\to\infty} \frac{e^{(k+1)}}{e^{(k)}e^{(k-1)}} = \frac{f''(x^*)}{2f'(x^*)}.$$

In fact, the secant method has order of convergence

$$\nu = \frac{1+\sqrt{5}}{2} = 1.6180\ldots.$$

Using the secant method for our implied volatility example gives

k	σ_k	$f(\sigma_k)$
0	0.2000000000000000	-3.54e-02
1	0.3000000000000000	8.26e-01
2	0.2041039868956179	-6.29e-04
3	0.2041769350923228	-1.09e-05
4	0.2041782248894078	2.25e-10
5	0.2041782248628699	-1.78e-15
6	0.2041782248628701	1.78e-15

Some simple examples - MATLAB `fzero` - Python `fsolve`

Here are some simple examples where Newton's method or the secant method can be used. In these examples, f and g are smooth functions.

Example

1. Given: g is a function and y is a value such that $g(x) = y$ has a unique solution x. Find x.
   $g(x) = y \Longrightarrow g(x) - y = 0$. Let $f(x) = g(x) - y$. Solve $f(x) = 0$.
2. Given: g is an invertible function and y is a value. Find $g^{-1}(y)$.
   $g^{-1}(y) = x \Longrightarrow g(x) - y = 0$. Let $f(x) = g(x) - y$. Solve $f(x) = 0$.
3. Given: f and g are two functions intersect at a point x_0. Find x_0.
   Let $h(x) = f(x) - g(x)$. Solve $h(x) = 0$ to find x_0.

Note: The MATLAB `fzero` solves the nonlinear scalar equation (1) using a hybrid method. In Python, `scipy.optimize.root_scalar` is the closest analogue; `scipy.optimize.fsolve` is intended for systems of nonlinear equations.

Systems of nonlinear equations

Suppose now that $\mathbf{r} : \mathbb{R}^n \to \mathbb{R}^n$ and that $\mathbf{x}^* \in \mathbb{R}^n$ is a solution of

$$\mathbf{r}(\mathbf{x}) = (r_1(\mathbf{x}), r_2(\mathbf{x}), \ldots, r_n(\mathbf{x}))^\top = \mathbf{0}.$$

Solving such a system of nonlinear equations is much harder than solving a single nonlinear equation.

Taylor expansion gives

$$r_i(\mathbf{x} + \mathbf{d}) = r_i(\mathbf{x}) + \sum_{j=1}^{n} \frac{\partial r_i}{\partial x_j} d_j + O(\|\mathbf{d}\|^2),$$

or equivalently,

$$\mathbf{r}(\mathbf{x} + \mathbf{d}) = \mathbf{r}(\mathbf{x}) + J(\mathbf{x})\mathbf{d} + O(\|\mathbf{d}\|^2), \tag{2}$$

where the Jacobian matrix is defined by

$$J(\mathbf{x}) = \begin{bmatrix} \partial r_1/\partial x_1 & \partial r_1/\partial x_2 & \cdots & \partial r_1/\partial x_n \\ \partial r_2/\partial x_1 & \partial r_2/\partial x_2 & \cdots & \partial r_2/\partial x_n \\ \vdots & \vdots & \ddots & \vdots \\ \partial r_n/\partial x_1 & \partial r_n/\partial x_2 & \cdots & \partial r_n/\partial x_n \end{bmatrix} = \begin{bmatrix} \frac{\partial \mathbf{r}}{\partial x_1} & \frac{\partial \mathbf{r}}{\partial x_2} & \cdots & \frac{\partial \mathbf{r}}{\partial x_n} \end{bmatrix}. \tag{3}$$

Newton–Raphson method (quadratically convergent !!!)

To generalize Newton's method, write

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}^{(k)}, \quad \text{how do we define } \mathbf{d}^{(k)}? \tag{4}$$

We want $\mathbf{x}^{(k+1)} \approx \mathbf{x}^*$, so by (2),

$$\mathbf{0} = \mathbf{r}(\mathbf{x}^*) \approx \mathbf{r}(\mathbf{x}^{(k+1)}) \approx \mathbf{r}(\mathbf{x}^{(k)}) + J(\mathbf{x}^{(k)})\mathbf{d}^{(k)}.$$

so we define $\mathbf{d}^{(k)}$ by requiring

$$\mathbf{r}(\mathbf{x}^{(k)}) + J(\mathbf{x}^{(k)})\mathbf{d}^{(k)} = \mathbf{0}. \tag{5}$$

Together, (4) and (5) define the Newton–Raphson method. Note that at each step we have to solve the $n \times n$ linear system

$$J(\mathbf{x}^{(k)})\mathbf{d}^{(k)} = -\mathbf{r}(\mathbf{x}^{(k)})$$

to obtain $\mathbf{d}^{(k)}$.

Newton–Raphson method - rectangular Jacobian

Assume that a function $\mathbf{r} : \mathbb{R}^n \to \mathbb{R}^m$, where $m \geq n$, is sufficiently smooth, and we want to find a solution of

$$\mathbf{r}(\mathbf{x}) = (r_1(\mathbf{x}), r_2(\mathbf{x}), \ldots, r_m(\mathbf{x}))^\top = \mathbf{0}.$$

This problem may not have a solution. Instead we can try to solve

$$\min_{\mathbf{x}\in\mathbb{R}^n} \|\mathbf{r}(\mathbf{x})\|_2^2 = \min_{\mathbf{x}\in\mathbb{R}^n} \mathbf{r}^\top(\mathbf{x})\mathbf{r}(\mathbf{x}) = \min_{\mathbf{x}\in\mathbb{R}^n} \sum_{i=1}^{m} (r_i(\mathbf{x}))^2.$$

Let $F(\mathbf{x}) = \|\mathbf{r}(\mathbf{x})\|_2^2$. Then we can apply the Newton-Raphson method for $\mathbf{f} = \nabla F = (\frac{\partial F}{\partial x_1}, \frac{\partial F}{\partial x_2}, \ldots, \frac{\partial F}{\partial x_n})^\top$, that is, we seek $\bar{\mathbf{x}}$ such that

$$\mathbf{f}(\bar{\mathbf{x}}) = \mathbf{0}.$$

Note: $\mathbf{f}(\bar{\mathbf{x}}) = \mathbf{0}$ gives a stationary point of F, and for a least-squares problem this is typically the minimizer we want.

With $\mathbf{f} = (f_1, \ldots, f_n)^\top$, equation (5) for $\mathbf{f}$ becomes

$$\mathbf{f}(\mathbf{x}^{(k)}) + J_{\mathbf{f}}(\mathbf{x}^{(k)})\mathbf{d}^{(k)} = \mathbf{0},$$

where $J_{\mathbf{f}}$ is the Jacobian matrix for the function $\mathbf{f}$. Now,

$$f_j = \frac{\partial F}{\partial x_j} = 2\sum_{i=1}^{m} r_i \frac{\partial r_i}{\partial x_j} = 2\frac{\partial \mathbf{r}^\top}{\partial x_j}\mathbf{r} \Longrightarrow \mathbf{f} = 2\left[\frac{\partial \mathbf{r}}{\partial x_1} \quad \frac{\partial \mathbf{r}}{\partial x_2} \quad \cdots \quad \frac{\partial \mathbf{r}}{\partial x_n}\right]^\top \mathbf{r} = 2J_{\mathbf{r}}^\top \mathbf{r},$$

where $J_{\mathbf{r}}$ is the Jacobian matrix for the function $\mathbf{r}$ (see (3)).

$$\frac{\partial \mathbf{f}}{\partial x_1} = 2\frac{\partial J_{\mathbf{r}}^\top}{\partial x_1}\mathbf{r} + 2J_{\mathbf{r}}^\top \frac{\partial \mathbf{r}}{\partial x_1} \approx 2J_{\mathbf{r}}^\top \frac{\partial \mathbf{r}}{\partial x_1} \Longrightarrow$$

$$J_{\mathbf{f}} = \left[\frac{\partial \mathbf{f}}{\partial x_1} \quad \frac{\partial \mathbf{f}}{\partial x_2} \quad \cdots \quad \frac{\partial \mathbf{f}}{\partial x_n}\right] \approx 2J_{\mathbf{r}}^\top \left[\frac{\partial \mathbf{r}}{\partial x_1} \quad \frac{\partial \mathbf{r}}{\partial x_2} \quad \cdots \quad \frac{\partial \mathbf{r}}{\partial x_n}\right] \approx 2J_{\mathbf{r}}^\top J_{\mathbf{r}}.$$

Therefore,

$$J_{\mathbf{r}}^\top(\mathbf{x}^{(k)})\mathbf{r}(\mathbf{x}^{(k)}) + J_{\mathbf{r}}^\top(x^{(k)})J_{\mathbf{r}}(\mathbf{x}^{(k)})\mathbf{d}^{(k)} = \mathbf{0}.$$

The last equation is just the normal equation of (5).

Approximating derivatives

Jacobian requires n^2 first-order partial derivatives of $\mathbf{r}$.

- Forward or backward difference approximation

$$\frac{\partial r_i}{\partial x_j}(\mathbf{x}) \approx \frac{r_i(\mathbf{x} + h\mathbf{e}_j) - r_i(\mathbf{x})}{h}, \quad \frac{\partial r_i}{\partial x_j}(\mathbf{x}) \approx \frac{r_i(\mathbf{x}) - r_i(\mathbf{x} - h\mathbf{e}_j)}{h}$$

  - $O(h)$ discretization error plus an $O(\epsilon/h)$ rounding error.
  - Minimize the total error $O(h + \epsilon h^{-1})$ implies $h \approx \sqrt{\epsilon}$.

- Central difference approximation

$$\frac{\partial r_i}{\partial x_j}(\mathbf{x}) \approx \frac{r_i(\mathbf{x} + h\mathbf{e}_j) - r_i(\mathbf{x} - h\mathbf{e}_j)}{2h}.$$

  - $O(h^2)$ discretization error plus $O(\epsilon/h)$ rounding error.
  - Minimize the total error $O(h^2 + \epsilon/h)$ implies $h \approx \epsilon^{1/3}$.